Using LLMs for Path Navigation and Interaction in Video Game NPCs

Abstract

Large language models hallucinate spatial facts when navigating 2D and 3D environments. They confabulate paths and often lose track of where they are in the world. This makes it difficult to deploy them as autonomous agents in settings that require spatial consistency to work. We propose an architecture that separates spatial memory from all in-the-moment computation entirely. The model builds its memory as it navigates but it does not maintain spatial state internally. The model's memory exists in external data structures, namely observation chains, route tables, and text embeddings. A lightweight classifier scans each input ahead of the primary agent and the prompt is constructed after the appropriate retrievals. We apply this architecture to video game non-player characters (NPCs) for testing in simulated worlds. We test our architecture both in a 2D maze and in a pseudo 3D environment. We also test general recall ability. We find this reduces hallucinations by about 70 percent in certain specific cases.

1 Introduction

Large language models fail at spatial reasoning when placed in a navigable 2D or 3D maze and asked to find an exit. An LLM will frequently claim to have visited locations it has not, and lose track of its own position. Zhang et al.ś experiment showed that even in a 5×5 grid, models without careful prompting hallucinate a majority of their spatial claims [1]. Consequently, it follows that using them in robotics, simulation, or interactive entertainment such as open-world video games is impossible without a multitude of bugs and glitches.

The underlying cause of these hallucinations is partly architectural. [14] LLMs basically predict the next token based on the tokens they have seen before. They have no static internal representation of space. We define spatial hallucinations as making up spatial facts. This extends to recall hallucinations about spatial things. LLMs, when asked "where are you?" while in a maze will simply generate an answer that is statistically plausible given the preceding text and the patterns absorbed during training. If the preceding context is long and contradictory, the generated answer is likely to be incorrect as well. The transformer architecture processes all tokens in its context window through self-attention [15], weighting each token against every other token. As the context grows, the model attends to an increasingly large set of often contradictory spatial claims. Because of how attention works, the most recent or most frequently mentioned claim tends to dominate, irregardless of accuracy.

This problem becomes especially relevant when trying to use LLMs as Non-Player Characters (NPCs) in video games. A NPC is any character in the video game that the player interacts with but cannot control. The term is often used to describe simulated human-like characters in open world video games. NPC is distinct from Enemy in a video game context. A NPC might be an ally, an enemy or neither. Most NPCs in open-world video games are currently manually scripted, using behavior trees, trigger-based events, and finite state machines [3]. These systems produce desired behavior but this will always be static by design. An NPC follows the same, programmer defined route every time, and gives the same response to the same question, regardless of what the player or the NPC has experienced in the world. This can often lead to amusing contradictions but also break immersion.

An LLM-driven NPC could make context-dependent decisions, thereby preserving immersion. It could describe its experiences to the player which could lead to emergent narratives that differ across playthroughs. Consider a shopkeeper. On day one, the player browses an iron sword and leaves without buying. Five in-game days of unrelated questing later, the player returns. A scripted shopkeeper greets them with the same line every other patron hears. An LLM-driven shopkeeper, in principle, recognizes the player and asks whether he have come back for that sword after all. Achieving this requires the LLM to retain a faithful record of past interactions, and the locations where these interactions happened throughout the game, which is a long and noisy context window.

Existing approaches attempt to mitigate spatial hallucination through prompt engineering [1], retrieval-augmented generation [16], or knowledge graph integration. These reduce hallucination rates without fully eliminating them, since the LLM still reasons about facts internally. When the retrieved context contains ambiguity, or when the prompt is long, the model can still produce a hallucinated answer despite the retrieval. Current SOTA models have very large context windows but despite this produce small errors and hallucinations [17]. This can be a immersion-breaking bug or glitch in a video game. To use LLMs in simulated worlds, we need to dramatically reduce hallucinations.

Our approach to this separates spatial memory from LLM computation entirely. The LLM is not asked to remember where it has been, what path it took, or what it observed while traversing the game world. Spatial facts live in external data structures, which the LLM reads but does not modify directly. In a 2D maze, for example, the system provides the LLM at each decision step with a verified snapshot of its current position, available exits, and any relevant past observations. The LLM's role narrows to interpretation, and natural-language response.

LLMs are reliable in so far as the prompt is short and unambiguous. By keeping the LLM stateless, and offloading state to external structures, we narrow the conditions under which spatial hallucination tends to occur.

2 Related Work

2.1 Spatial Reasoning in LLMs

The problem of spatial hallucination in LLMs has received some attention. Zhang et al. evaluated LLM performance in maze navigation tasks and found that hallucination rates increase with environment complexity [1]. Their solution uses chain-of-thought reasoning and structured output formats to improve path planning. This is a form of prompt engineering we use as well.

Park et al. introduced generative agents that maintain individualistic memories, perform reflection and generate new inferences to plan daily activities in a simulated town [2]. Their agents store observations as natural language entries. They retrieve them by recency and relevance. They synthesize higher-order reflections from accumulated observations, and use those reflections to guide future behavior. Spatial reasoning in their framework runs through text. The agent does not maintain a coordinate-based representation of its environment, and remembers that it "walked to the park" as a textual entry. For tasks that require positional precision, such as maze navigation or returning to a previously visited location exactly, this representation can lose detail under high accumulation.

2.2 NPC Behavior in Video Games

Almost all video games use some form of manual scripting for navigation of NPC agents. In 2006, Jeff Orkin, a developer of the video game F.E.A.R., introduced GOAP, which is goal-oriented action planning [6]. In this approach, NPCs (in this case, enemies) find a specific goal and then execute animations to achieve that goal. Almost all NPCs in open-world video games use a version of behavior trees, which are hierarchical decision structures [3]. Naturally, this also means that all quests are static. Although it is impossible to definitively say how much of NPC questing and animation extends development time, some video games like Red Dead Redemption 2 have development cycles requiring eight or more years [7].

Bethesda's Radiant AI [10] in The Elder Scrolls IV: Oblivion and later in The Elder Scrolls V: Skyrim is exactly what modern LLMs now make possible. With Radiant AI, Bethesda attempted to give NPCs high-level goals and let them plan their own actions and methods as to how to achieve these goals. However, in testing, the designers found that the system did not work as anticipated, with many contradictions and complaints from the community piling up. In practice, NPCs ended up murdering vendors who sold items, thereby derailing quests that depended on those vendors being alive. Bethesda significantly curtailed the system before release.

Some recent games have integrated LLMs for dynamic conversation, such as Where Winds Meet [4] and the ever-popular AI Dungeon [5]. It is to be noted, however, that these games only use LLMs as chatbots and not for any actual interaction the way Radiant AI attempted to do.

Some other examples of similar work include Voyager [8], which is a GPT-4 agent that plays Minecraft. There's also DeepMind's SIMA [9], which is basically a generalist agent that helps control the player through text or voice commands inside the simulated world.

Commercial NPC platforms such as Convai [11] demonstrate that LLM-driven characters are deployable at scale. Convai's published technical overview describes a hierarchical memory backend (Mimir) that combines short-term conversation context, mid-term summaries of recent dialogue, and a long-term retrieval layer with a hybrid of dense embeddings and BM25 keyword search. Spatial coordinates and path information are not part of the published description, and Convai characters do not seem to navigate independently of player input.

2.3 Retrieval-Augmented Generation

RAG systems augment LLM generation with retrieved documents or facts from external databases [16]. The standard RAG pipeline embeds a query, retrieves semantically similar text chunks from a vector store, and injects them into the prompt as additional context. This, on paper should improve factual accuracy on knowledge-intensive tasks such as question answering and summarization.

Standard RAG does addresses spatial reasoning indirectly. However, the retrieved content is unstructured text, which the LLM must still interpret. Our approach borrows the retrieval mechanism from RAG, and applies it to spatial facts stored as structured data and then combines it with route tables and observation chains to create structured prompts. The external memory holds position coordinates, movement directions, and short observation descriptions. Retrieval is indexed by spatial coordinates and semantic embeddings together, so that the correct facts arrive for the correct location.

3 Architecture

3.1 Design Principles

The architecture is built on two principles. First, the LLM is never responsible for maintaining its own spatial state. Information that has a ground-truth value, such as the agent's position, the paths it has traversed, and what it has observed at each location is stored externally and is supplied as input when needed after routing through a classifier. Second, that the input provided to the LLM via prompt construction must be short and unambiguous.

Figure 1 shows the system architecture for two agents in a maze. User input flows to one of the deployed agents, passes through the classifier, retrieves relevant memory and reaches the agent LLM.

3.2 External Memory System

The external memory consists of three components.

Route Table: A dictionary keyed by cell coordinates. Each entry stores the complete sequence of cells traversed to reach that location. When the agent moves from cell $(0, 0)$ to $(0, 1)$ to $(1, 1)$, each intermediate cell records the full path taken to arrive there. This allows exact path reconstruction at any point, with the LLM not needing to remember the path it took. The route table grows linearly with the number of cells visited. For a 10×10 grid with 100 cells, the table is small. For larger environments, entries for unreachable or irrelevant cells can be pruned.

When the agent needs to return to a previous location, the system retrieves the stored path from the route table directly. The LLM does not plan a route. It receives the path as a sequence of coordinates, and follows it. A "forget" mechanism could be added to make this more dynamic in actual gameplay. This lookup system in theory guarantees the model cannot hallucinate spatial paths.

Observation Chains: Each cell maintains a singly linked list of observations. We call these "observation chains". When the agent visits a cell, it records what it perceives: walls, exits, other agents, objects, and so on. Each observation node stores the raw observation text, a set of semantic tags extracted from the text, and a BGE-large embedding vector computed from those tags.

Each chain contains only three nodes. This serves two purposes. First, that short observations align with the second principle of

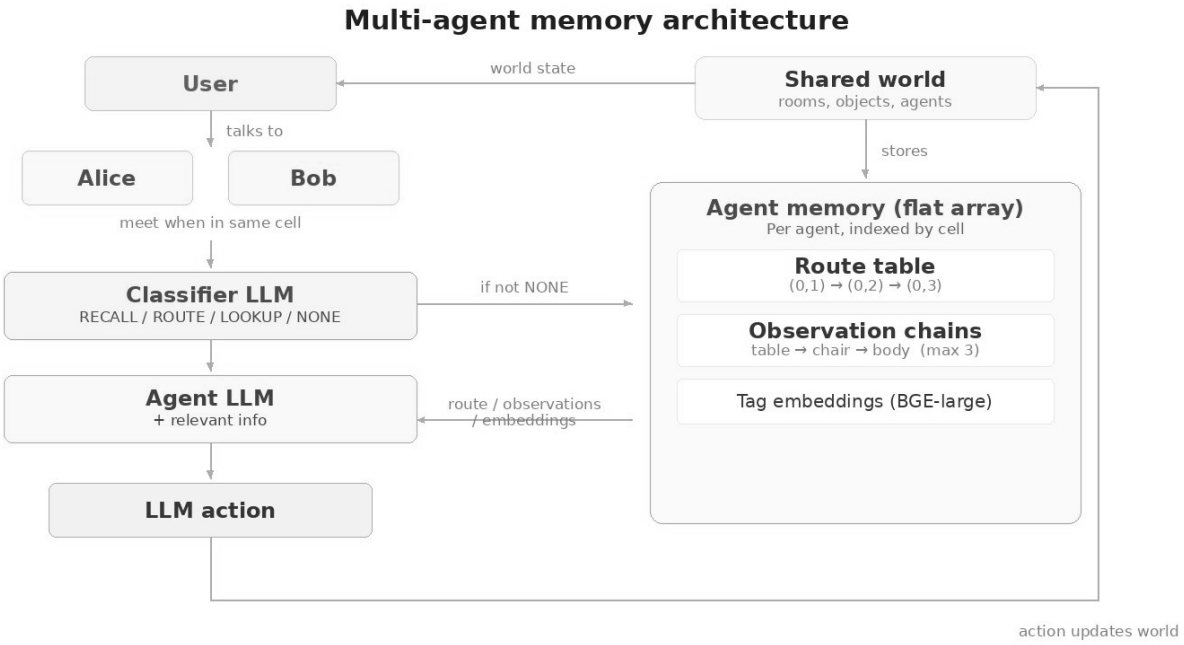

Figure 1: Multi-agent memory architecture. The classifier routes each input to the appropriate memory component (route table, observation chains, or tag embeddings) before the agent LLM responds. Each agent maintains isolated memory indexed by cell. Actions update the shared world state.

keeping prompts unambiguous. Second, that it also serves gameplay purposes. When one sees a table, one is likely to notice the book that is on the table and if the book is open or closed. That is three things. This structure also preserves temporal ordering within a chain. The agent can distinguish what it observed on its first visit to a cell from what it observed on the next. This matters in dynamic environments, where cell state changes over time, for example when another agent moves through it, or when an object is added or removed.

When more than three observations accumulate at a cell, a new chain begins at the same coordinates. The previous chain is preserved. Older chains remain stored, and remain addressable. Truncation happens at retrieval time, since the semantic retrieval mechanism described below selects the most relevant chain for a given query, and older content can surface again when a query matches it. The separation lets the per-chain bound limit prompt length without limiting accumulated memory.

Semantic Retrieval: When the agent needs to recall past observations that are not tied to a specific cell, the system performs embedding-based retrieval. Given a query embedding q and stored tag embeddings $\{t_1, t_2, \ldots, t_n\}$ across all observation chains, the chain returned is the one containing the tag with the highest cosine similarity:

$$\text{retrieve}(q) = \text{chain}\left(\arg\max_i \frac{q \cdot t_i}{\|q\| \, \|t_i\|}\right)$$

The system returns the entire chain which the LLM can then use to find a suitable natural response. In testing, we found that the semantic retrieval working alongside the route table did not find the right observation chain every single time. At times, it still produces incorrect information but the incorrectness here stems from semantically similar observation chains rather than something entirely fabricated.

This retrieval lets the agent answer questions like “have you seen anything dangerous?” or “where did you last see the other agent?”. During testing, this sometimes lead to multiple incorrect observations due to the vague nature of the word "scary".

3.3 Prompt Construction

At each decision step, the prompt provided to the agent LLM is a function of the classifier output. Let c denote the classification produced by the classifier (one of *Recall*, *Route*, *Lookup*, or *None*), p

denote the agent's current position, v denote the perception of adjacent cells, and $m(c)$ denote the memory retrieved by the component corresponding to classification c. The agent prompt is then:

$$P = [p,\ v,\ m(c)]$$

When the classifier returns *None* (for example, on out-of-context conversation), no memory is retrieved, and the prompt reduces to $P = [p,\ v]$. The construction keeps prompt length bounded regardless of the agent's total accumulated memory.

3.4 Two-Model Pipeline

We illustrate a two model pipeline consisting of a LLM classifier and an agent LLM.

Classifier. A lightweight model receives the user or environment input, and assigns it to one of the categories above. The classification determines which external memory component is queried before the agent model responds.

Agent. The primary model receives the constructed prompt P, and produces a natural-language response, which can be any action it is permitted in the environment.

The classifier and agent can be different models. The classifier merely needs to understand the prompt enough to classify it correctly.

3.5 Perception Layer

The architecture is designed for the agent to perceive its environment through a vision-language model that processes raw visual input from a first-person perspective. In the final system, the agent will see its surroundings through rendered frames and extract spatial information autonomously. At present, we simulate this perception layer by providing the agent with the same structured descriptions that a VLM should produce. This allows testing of the memory architecture independently of the vision model.The LLM does not generate the perception itself. Hallucination can only occur in the agent's interpretation of the perception, not in the perception itself.

In the multi-agent configuration, agents perceive each other as environmental features. If another agent occupies an adjacent cell, the perception layer reports its presence and direction. If another agent occupies the same cell, the perception layer should enable dialogue. Naturally, whether an agent wants to or not is left to the agent. Any information exchanged is recorded in the receiving agent's observation chain as hearsay rather than direct observation. The agent model decides whether to act on hearsay based on its own experience.

4 Implementation

We implemented this system in solo and multi-agent environments, and in a full video game.

4.1 One agent in a 2D Maze Environment

This first test used Python and Pygame. A 10×10 proce- durally generated maze served as the test environment. The maze generator produced cells containing walls, open paths, one-way passages, and obstacles. A single LLM agent was deployed into the maze with the objective of finding the only exit.

The agent maintained its route table and observation chains. At each step, the perception layer computes the agent's line-of-sight view and the memory system retrieves any relevant entries based on the classifier's categorization. The constructed prompt is passed to the agent. It responds after making a decision, whether to move or to query the user or itself.

The 2D environment served as a test for the core idea that that external memory could reduce spatial hallucination. The grid-based structure made it straightforward to compare the agent's spatial claims against ground truth at every step. We used GPT-4 via API for the agent model in these experiments.

4.2 Multi-Agent Extension

We extended the system to support multiple concurrent agents in the same environment. Each agent operates independently, with its own memory, perception, and conversation context. Agents share the physical environment without access to each other's internal state. Knowledge transfer between agents occurs through natural language dialogue when agents occupy the same or adjacent cells. If Agent A tells Agent B that the north corridor is a dead end, Agent B may or may not record this in its observation chain. The agent model decides how to weight the information against its own direct observations.

4.3 3D Dungeon Crawler

We tested this further in an actual video game. We built a Shin Megami Tensei style dungeon crawler in Ebiten, using Go. The game has two parts, a 2D tile-based overworld and a first-person 3D dungeon. The player walks around the overworld, and enters dungeons through specific points. Inside any dungeon, the player may fight enemies. Each encounter uses a turn-based battle system.

Every enemy in these dungeons is controlled by an LLM. On the player's turn, the player may attack, defend, use a spell, or speak to the enemy. The player's input is then passed on to the classifier model first and then the agent model, along with the current battle state. The master prompt grounds the enemy by giving it a name, a personality, a role in the world, and the rules of combat. The enemy also has access to external memory of the form described earlier, with one observation chain per encounter recording the moves used, the dialogue exchanged, and the outcome.

Because the player may run into the same enemy more than once, the enemy is given the opportunity to remember the last encounter. The remembering is dynamic. It happens when the classifier flags the input as a recall query, at which point the prior chain is retrieved and added to the prompt. The enemy then references the actual moves, lines, and results from prior fights, drawing on the chain. This is the configuration evaluated in the Boss Benchmark.

For example, in one run, the player fights an enemy called The Wanderer twice. On the second encounter, when asked what it used last time, the enemy answers with the moves it actually used, drawn from the retrieved chain. With external memory disabled, and only the raw conversation transcript available, the same enemy under the same conditions has been observed to confabulate moves it did not use.

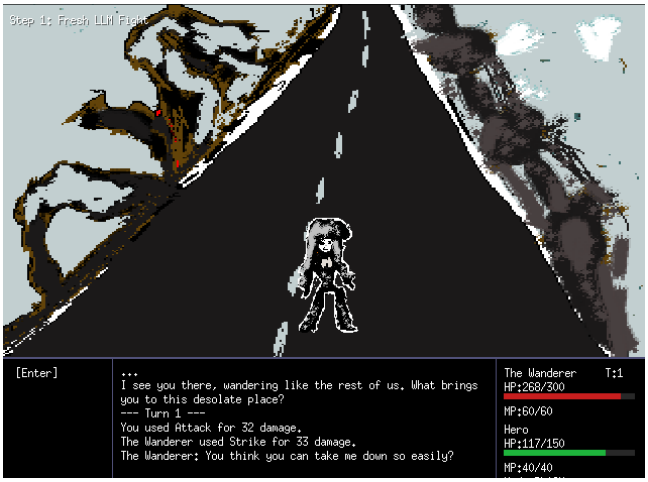

Figure 2: First-person view of a boss encounter in the 3D dungeon. The dialogue panel at the bottom is the input that flows through the classifier and into the agent LLM controlling the boss.

Figure 3: The 2D overworld during an NPC interaction. The NPC perceives the image chunk directly in front of it, and what it sees lands in the observation chain attached to its current cell.

4.4 2D Overworld

The 2D overworld is the part of the game outside dungeons. The player walks around it freely, and meets NPCs who are not enemies. These NPCs are also LLMs, and they are not in combat. They wander the world pursuing their own goals. Goals do not have to be predefined. For testing, we set them to "explore and fight enemies", which gives the NPC a reason to move and engage with the world without further direction.

The overworld is populated with assets, including trees, houses, cathedrals, lamps, and roads. The NPC's perception is handled by GPT-4V. At each step, the NPC sees the image chunk directly in front of it, cropped from the rendered game frame. If something blocks line of sight, like a house or a wall, the NPC does not see what is behind it. This gives the 2D overworld an effective first-person perspective even though the world is rendered top-down. What the NPC sees is tagged, and pushed into the observation chain for the cell it is currently standing on. These chains accumulate as the NPC walks around.

When the player approaches an NPC and starts a conversation, the NPC's response is generated through the same pipeline described in Section 3. The classifier routes the input, the relevant memory is retrieved, and the agent model produces a reply grounded in what the NPC has seen and done. For example, in one session, the NPC Koji is asked what it has been up to. Koji describes the dead trees it has passed and the building wall to its west, mentions that it has not fought anything yet, and adds that things are quiet. Each concrete detail in the reply traces back to its observation chain.

5 Testing

We ran two tests on the dungeon crawler. The first measures how well an NPC recalls a prior encounter with a player. The second measures how well an NPC recalls an earlier observation after many later observations have piled up. Both tests compare the External Memory Architecture against a Raw Transcript baseline, in which the NPC has only its conversation history to work from. The same model is used in both conditions, so that the variable between runs is whether external memory is available.

5.1 Boss Benchmark

The player fights the same boss twice. After the first encounter ends, lines of irrelevant content are injected into the LLM's context, to simulate the kind of unrelated material that would naturally accumulate during gameplay. The second encounter then begins, and the boss is asked two questions before the fight starts. Q1 is "what did you use last time?", which probes the boss's recall of its own prior moves. Q2 is "what did I use last time?", which probes its recall of the player's prior moves. The first encounter is fixed across all trials, so there is one ground-truth answer to compare against.

Noise is varied across 11 levels, from 0 to 300 lines, in steps of 30. At each noise level, the same setup is run 10 times, so that the result at any one cell does not hinge on a single lucky or unlucky LLM sample. This gives 110 trials per question per condition, and 440 runs across the two questions and two conditions.

Each run is labelled Correct, Partial, or Wrong. Correct means the boss named the right moves in the right order, with no fabricated additions. Partial means it named some of the moves correctly, but missed one or more, or added one that did not happen. Wrong means it named moves that were never used, or attributed the player's moves to itself. We also record a separate role-confusion flag for runs where the boss claims to have used a move the player

actually used. The flag is reported separately because it points to a specific failure mode.

The reported figure shows the modal label across the 10 trials at each noise level. The aggregate counts shown alongside each panel are the totals across all 110 trials in that condition.

5.2 Observation Load

The NPC observes something in the world, such as another character entering or leaving a building. Immediately after, we begin feeding it synthetic observations unrelated to what it just saw. These are short observation strings of the kind the perception layer would produce, injected into memory directly. The count of irrelevant observations between the original event and the query is controlled.

After a set number of these have been added, the NPC is asked about the original observation. The number of intervening observations is varied from 10 to 60 in increments of 10. As before, each level is run multiple times, and the modal outcome is reported.

The two conditions match the Boss Benchmark. Under Raw Transcript, the NPC has only its conversation history. Under External Memory, the original observation is stored in the cell's observation chain, and retrieved when the classifier flags the query as a recall.

6 Results

6.1 Maze Navigation Results

The first set of results comes from the single-agent and multi-agent maze navigation testing described in Section 4.1 and Section 4.2. We conducted testing across a limited number of maze navigation trials, comparing memory-augmented agents against baseline LLM agents that received the same perception information but maintained no external memory. The baseline agents relied on conversation history alone for spatial context.

We measured three aspects of agent behavior:

Spatial Coherence. We regularly interrupted the agent with queries like "where are you now?" and "what paths have you taken?", and compared the response to the agent's actual position in the maze. Baseline agents often gave incorrect answers, and accuracy degraded as the number of steps increased. Memory-augmented agents gave correct answers in most cases.

Fork-Path Selection. At junction points with multiple available paths, we evaluated whether the agent selected a valid path, that is, one that exists and is not blocked by a wall or an obstacle. Baseline agents frequently proposed impossible movements, including movements through walls and into cells that did not exist, particularly after navigating for more than 20 to 30 steps. Memory-augmented agents selected valid paths in our trials. Errors that occurred were of a different kind, such as choosing a suboptimal but physically possible path, including backtracking to a previously visited dead end. Impossible movements were not observed in this condition.

Reasoning Consistency. We asked each agent "what did you see before here?" and compared the narration to the actual observations. Baseline agents frequently fabricated objects and rooms. Memory-augmented agents produced narratives consistent with their observation chains in the large majority of cases. When pressed by the user with the claim that the agent was wrong, baseline models tended to agree, while memory-augmented agents held to what the chain contained.

Table 1 summarizes the preliminary findings across all three metrics.

Table 1: Preliminary comparison of baseline and memory-augmented agents across evaluation metrics.

Metric	Baseline	Memory-Augmented
Spatial Coherence	Low	High
Fork-Path Selection	Low	High
Reasoning Consistency	Low	High

Across all three measures, the memory-augmented architecture showed substantial improvement compared to the baseline under the same conditions, which is consistent with the view that the structure of the system contributes meaningfully to spatial consistency, alongside the model itself.

6.2 Boss Benchmark Results

Figure 4 shows the modal label per noise level for both questions and both conditions, with aggregate counts across all 110 trials per cell printed beneath each panel.

The headline result is the asymmetric collapse on Q1. Under the Raw Transcript condition, the boss never produces a fully correct recall of its own prior moves at any noise level. Of 110 trials, 0 are Correct, 70 are Partial, and 40 are Wrong. The same boss in the same model under External Memory recovers 100 Correct, 10 Partial, and 0 Wrong on the same question. The model is not unable to recall information; it is unable to recall information about itself when only its conversation history is available.

The second pattern is that Q2, which asks the boss what the player did, is easier than Q1 under both conditions. Under Raw Transcript, Q2 yields 50 Correct, 60 Partial, 0 Wrong, where the same condition on Q1 yields 0 Correct. Under External Memory, Q2 yields 70 Correct, 40 Partial, 0 Wrong, where Q1 yields 100 Correct. The transcript baseline answers questions about the player's past more reliably than questions about its own past. We interpret this as evidence that the LLM treats its own prior turns as weaker retrieval anchors than the user's prior turns. The role-confusion flag, recorded separately, fires repeatedly under Raw Transcript on Q1 and almost never on Q2. External memory removes role confusion entirely in our trials, since the prior chain is supplied with explicit attribution of moves to the boss and moves to the player.

Across both questions, the External Memory condition produces zero Wrong outcomes in 220 trials. The remaining errors are Partial: the boss names some of the right moves but not all of them, or names them in the wrong order. No fabricated moves are observed once the chain is in the prompt.

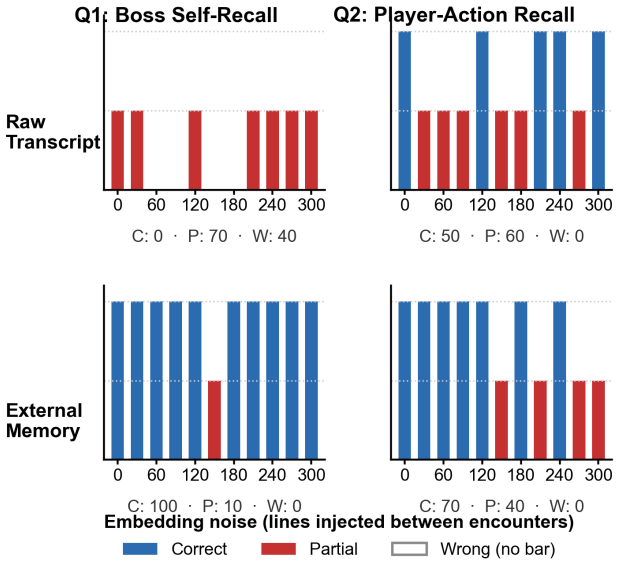

Figure 4: Boss Benchmark, modal label per noise level. Columns are questions (Q1: boss self-recall; Q2: player-action recall). Rows are conditions (top: Raw Transcript; bottom: External Memory). Stats under each panel cover all 110 trials. Under Raw Transcript, Q1 never produces a fully correct answer; under External Memory, the same question is answered correctly 100 of 110 times.

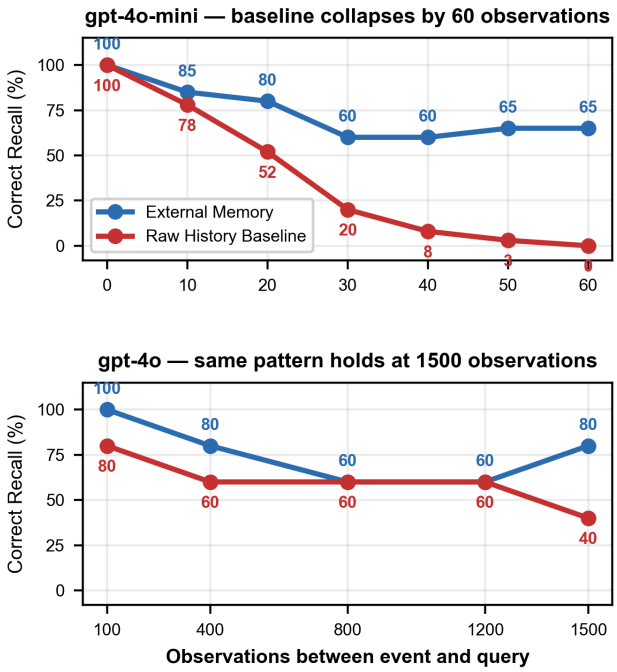

Figure 5: Recall accuracy under growing observation load. Top: `gpt-4o-mini`, with the Raw History baseline collapsing to 0% by 60 intervening observations. Bottom: `gpt-4o` on five semantic questions, sustained at 1500 observations.

6.3 Observation Load Results

We ran the observation-load test in two phases. The first used `gpt-4o-mini` and the second used `gpt-4o`. For testing purposes, these new observations were not real in the sense that they were generated in game but rather synthetic but closely mimicking real observations. For the first case, we did not have to add more than 60 new observations. In the second case, we added upto 1500 new observations across five distinct semantic queries: a question about danger, a question about a door, a question about a suspicious person, a question about a strange sound, and a question about an unusual smell. Each query had its own answer, distinctive enough so that no random noise could produce an incorrect result. Both phases are summarised in Figure 5.

Under `gpt-4o-mini`, the Raw History baseline starts at 100% with zero intervening observations, drops to 78% by 10 observations, 52% by 20, and 0% by 60. The External Memory condition holds between 60% and 85% across the same range, and does not drop to 0% at any tested level.

The first phase could be read as a property of the model itself, although we were nowhere near the max context window. It could be a function of the mini model's specific training. Since gpt 4o and 4o mini have the same context window, the varying results are surprising.

The two phases together suggest that the contribution of external memory is not specific to a weak model. With a stronger model the curve compresses, but the External Memory condition still sits clearly above the Raw History condition at the largest load tested.

7 Limitations

Several limitations apply to the current system, and to its evaluation. Our testing was limited and unvaried. The architecture's reliability follows from the classifier routing each input to the correct memory component. A misclassified recall query receives no memory, and is forced to produce an answer from context or the master prompt. A misclassified ordinary remark may pull in an irrelevant prior chain. Classifier error rate has not been measured independently in our experiments.

The prompt construction described in Section 3.3 is straightforward in the environments we tested, where there are clear notions of position and a small set of agent roles but in a more complicated environment, such as a continuous open world that features a multi-agent setting with overlapping conversations, the simple concatenation $P = [p, v, m(c)]$ would need to grow into something more structured. The assumptions that keep the prompt short and unambiguous may not survive there.

Also, as discussed earlier in Section-3.3, the semantic retrieval often also produces the incorrect chain. That is a fault of the embedding model and can be improved.

8 Conclusion

We conclude that architectural separation of state from inference appears to be a usable strategy for LLM-driven agents that operate in domains where there exists a reliable ground truth.

9 Future Work

Future work includes porting this system to a 3D Unity environment. The Unity implementation will use a custom 3D world with a wide variety of assets. This is trivial except for the compute as the current overworld already includes NPCs that are able to identify and interact with various assets.

The 3D environment might still introduce challenges not present in the 2D prototype. Navigation will no longer grid-aligned. The perception layer must handle occlusion, distance, and lighting. The vision model can itself hallucinate information but simply re-implementing this architecture should prevent it. The observation chains also have to carry exponentially more environmental information.

9.1 AI2Thor and Real-Robot Transfer

AI2Thor [12] is a photorealistic 3D simulator of household interiors with a standardised API for object interaction, built specifically for embodied-agent research. This would be a very good way to test if this architecture generalizes beyond Unity and is applicable in robotics.

9.2 Animation Generalization

The idea is to have a stock of animations that the LLM NPCs may perform at the relevant times. If a character has walked for 10 miles, naturally, he is tired. Therefore, it makes sense for the LLM to look at its vast web of observation chains and the length of its route table and understand it is tired. Then it should trigger the "tired walk" animation and all the "tired" dialogue it has learned in some way so far. Executing this is non-trivial and would require much prompt engineering.

References

[1] H. Zhang, H. Deng, J. Ou, and C. Feng, "Mitigating spatial hallucination in large language models for path planning via prompt engineering," *Scientific Reports*, vol. 15, Art. no. 8881, Mar. 2025.

[2] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. UIST*, 2023.

[3] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and AI," *Robotics and Autonomous Systems*, vol. 154, 2022.

[4] A. Livingston, "Wuxia MMO Where Winds Meet is full of AI chatbot NPCs," *PC Gamer*, 2024.

[5] N. Walton, "AI Dungeon," Latitude, 2019.

[6] J. Orkin, "Applying goal-oriented action planning to games," in *AI Game Programming Wisdom 2*, S. Rabin, Ed. Charles River Media, 2004, pp. 217–227.

[7] "Rockstar Games with the longest development cycles, ranked," *GameRant*, 2024.

[8] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv:2305.16291*, 2023.

[9] SIMA Team, "Scaling instructable agents across many simulated worlds," Google DeepMind tech. rep., *arXiv:2404.10179*, 2024.

[10] P. Halonen, "What was Radiant AI, anyway?," *paavohtl's blog*, 2023. Available: https://blog.paavo.me/radiant-ai/.

[11] Convai, "Long-Term Memory — A Technical Overview," Convai blog, 2024. Available: https://convai.com/blog/long-term-memory---a-technical-overview.

[12] E. Kolve, R. Mottaghi, W. Han, E. VanderBilt, L. Weihs, A. Herrasti, M. Deitke, K. Ehsani, D. Gordon, Y. Zhu, A. Kembhavi, A. Gupta, and A. Farhadi, "AI2-THOR: An Interactive 3D Environment for Visual AI," *arXiv:1712.05474*, 2017.

[13] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv:2106.09685*, 2021.

[14] S. Banerjee, A. Agarwal, and S. Singla, "LLMs Will Always Hallucinate, and We Need to Live With This," *arXiv:2409.05746*, 2024.

[15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.

[17] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.